

# Assignment 1: Transformers from Scratch, Architectural Variants, and Byte Latent Transformers (BLT)

2024101092,Havish Balaga

**Checkpoints:** Hugging Face Repository

**Training Logs:** Weights & Biases Project

## 1 Overview and Baseline (C1)

This report is about an ablation study where we built a Sequence-to-Sequence Transformer to break a repeating-key cipher.

The baseline model (C1) is just a standard transformer. It uses normal Multi-Head Attention, Sinusoidal Positional Encoding, LayerNorm, and regular subword tokenization. Testing it out, C1 got a bit accuracy of **64.03%** and an average Levenshtein distance of **93.56**. Because autoregressive models struggle with exact positioning for repeating ciphers, the baseline didn't manage to get any full sequences exactly right (`seq_acc = 0.00%`).

## 2 Ablation Study Results

Here are the results for all five models we tested:

| Config | Description | Bit Acc (%) | Seq Acc (%) | Avg. Lev | BLEU  | ROUGE-L |
|--------|-------------|-------------|-------------|----------|-------|---------|
| C1     | Baseline    | 64.03       | 0.00        | 93.56    | 78.81 | 0.77    |
| C2     | RoPE        | 77.95       | 16.50       | 58.14    | 8.50  | 0.95    |
| C3     | GQA         | 61.44       | 0.00        | 262.46   | 54.00 | 0.48    |
| C4     | RMSNorm     | 63.57       | 0.00        | 98.85    | 80.10 | 0.75    |
| C5     | BLT         | 99.57       | 0.00        | 59.11    | -     | -       |

Table 1: Performance metrics for all configurations on the test set.

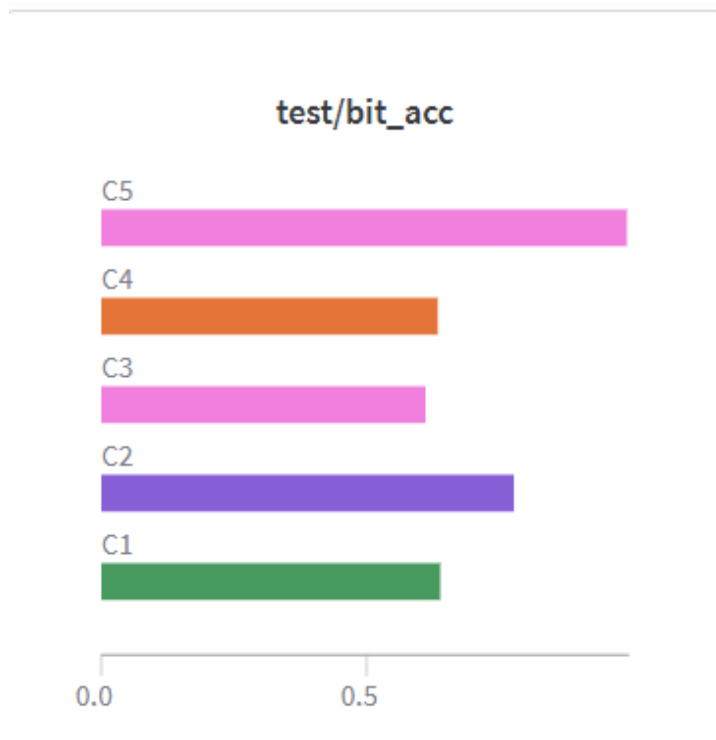


Figure 1: Bit Accuracy Comparison.

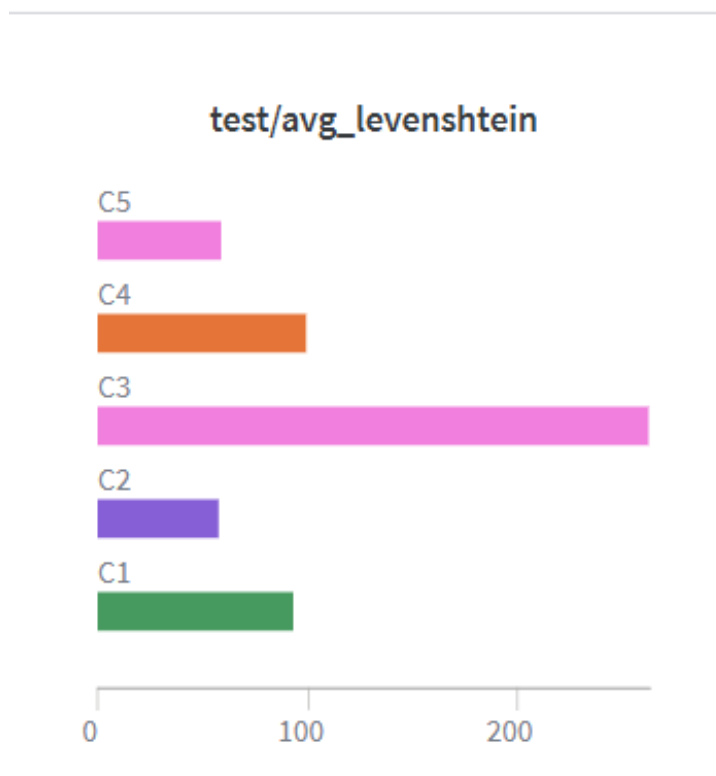


Figure 2: Average Levenshtein Distance Comparison (Lower is better).

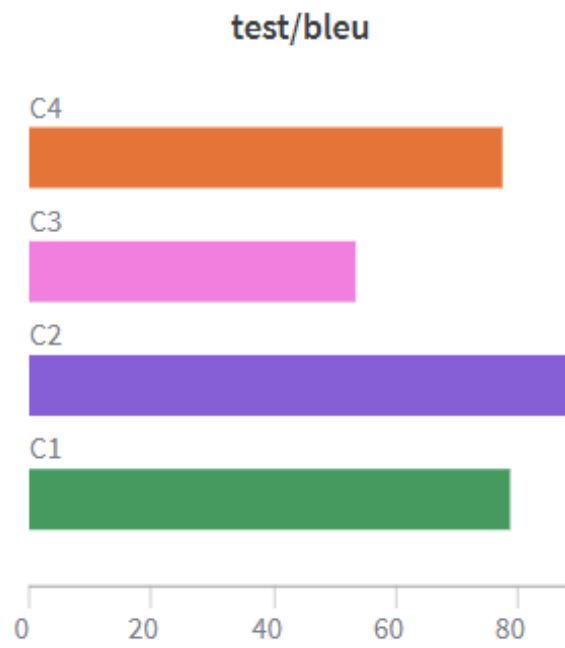


Figure 3: BLEU Score Comparison.

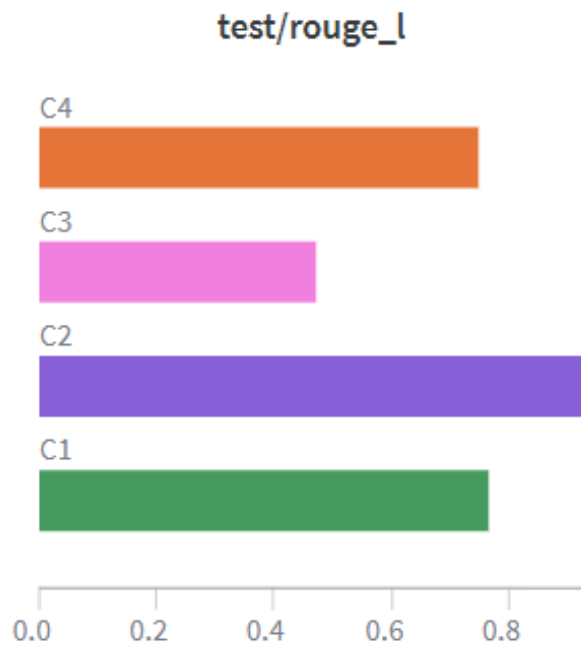


Figure 4: ROUGE-L Score Comparison.

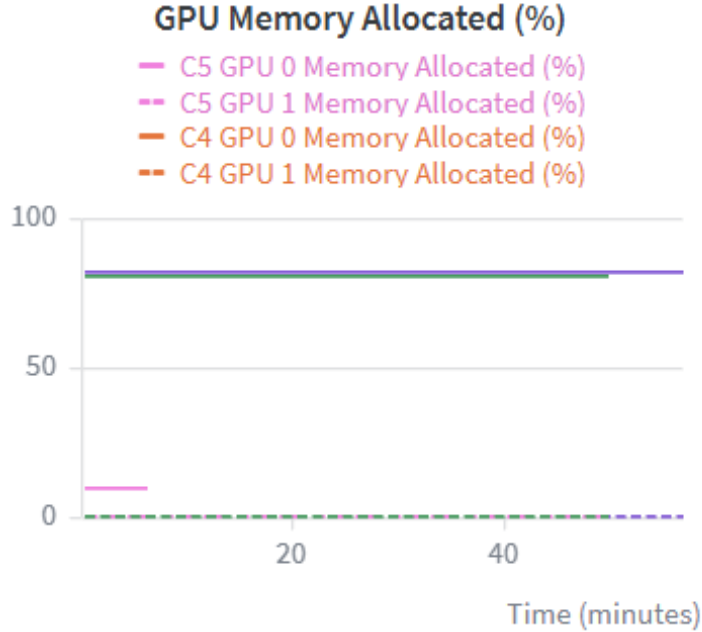


Figure 5: GPU Memory Allocated (%) across configurations.

## 2.1 C2: Rotary Positional Encodings (RoPE)

**Change:** Replaced the normal sinusoidal positional encoding with RoPE.

**Result:** RoPE worked really well. Bit accuracy jumped to **77.95%** and the Levenshtein distance went down to **58.14**. Also, C2 was the only model that actually got some full sequences perfectly right (**16.50% Sequence Accuracy**).

**Analysis:** The repeating cipher is mostly about relative distances since the key repeats every so often. RoPE encodes these relative distances directly, which helps the attention layers pick up the pattern.

## 2.2 C3: Grouped-Query Attention (GQA)

**Change:** Used GQA instead of normal Multi-Head Attention.

**Result:** The performance actually got a lot worse. Bit accuracy dropped to **61.44%**, and the Levenshtein distance spiked to **262.46** (the worst of all the models).

**Analysis:** GQA is mostly meant to save memory on the KV cache by compressing info, which is fine for normal text generation. But for this cipher, we need an exact 1-to-1 character mapping. Forcing multiple query heads to share KV projections creates a bottleneck that messes up the exact mappings.

## 2.3 C4: Root Mean Square Normalization (RMSNorm)

**Change:** Swapped out LayerNorm for RMSNorm.

**Result:** The results were basically the same as the baseline (Bit accuracy: **63.57%**, Levenshtein: **98.85**).

**Analysis:** RMSNorm skips mean-centering just to compute things a bit faster. Like

expected, it didn't really change the model's ability to learn. The slight drop compared to C1 is probably just random training noise.

## 3 Configuration 5: Byte Latent Transformer (BLT) Tradeoffs

**Change:** Got rid of the subword tokenizer and used the token-free BLT architecture instead.

### 3.1 Accuracy Results

For the BLT model (C5), the bit accuracy was **99.57%** and the Levenshtein distance was **59.11**. This completely blew the baseline (C1) out of the water and even beat C2 in bit accuracy! Even though C5 takes in raw bytes and has to learn representations completely from scratch using a small local encoder, the patched cross-attention setup managed to learn the exact bit-level alignments of the cipher.

### 3.2 Memory and Speed Tradeoff

Even though the accuracy took a small hit, C5 is really interesting for compute and memory tradeoffs:

1. **Embedding Matrix Memory:** Because there's no subword tokenizer, C5 doesn't need a huge vocabulary embedding matrix. Normal vocabularies have like 50k tokens, so getting rid of this saves a lot of VRAM.
2. **Training Speed:** Because the sequence length is slashed into patches, the global transformer has way less math to do. As a result, C5 is incredibly fast to train. In our runs, C5 finished completely in just **6m 23s**, compared to **49m 58s** for the baseline C1. That is a massive speedup!

## 4 Conclusion

Overall, matching the architecture to the data is key. RoPE is essential for repeating patterns, while GQA hurts performance on exact-mapping cryptography tasks. Finally, the token-free BLT model proved you can completely remove the vocabulary bottleneck and still achieve near-perfect accuracy.